Design Pattern Analysis in a FastAPI-Based Logistics Management Application

Warehouse & Transportation Management System (WTMS)

Expanded Case Study Report

Baseline analysis • Design evolution • Cross-language implementation • Experimental evaluation

This report separates findings supported by inspected source code from proposed refactorings and measured results. The four-language programs are small executable demonstrations, not replacements for the complete WTMS backend.

Abstract

This case study examines a FastAPI-based Warehouse & Transportation Management System (WTMS) backed by PostgreSQL. The work is organized into three parts: identifying patterns present in the baseline source, analyzing how the design should evolve as logistics requirements change, and comparing a representative pattern implemented in Python, Java, JavaScript, and C++. The baseline analysis confirms a Repository-based data-access structure with four repository instances. The evolution work adds delivery-partner support, shipment-tracking records, and transactional route-change history. The cross-language Repository Pattern examples passed their functional checks, and process-level execution times were measured using a consistent local procedure. Results are interpreted cautiously because the programs are small and measurements include process startup overhead.

1. Introduction and Objectives

Logistics systems coordinate inventory, warehouses, orders, vehicles, drivers, routes, and shipments. As requirements grow, code that combines HTTP handling, business decisions, database access, and external integrations can become difficult to test and extend. Design patterns provide reusable structures for separating these responsibilities, but a pattern should only be reported as present when the implementation provides evidence for it.

The objectives are to inspect the actual WTMS source; identify patterns supported by code evidence; analyze new logistics requirements and suitable refactorings; preserve the existing repository abstraction; implement one representative pattern in four languages; and record tests and measurements without inventing results.

2. Application Overview

The application uses FastAPI for HTTP APIs and PostgreSQL for persistent storage. Inspected feature areas include warehouse management, multi-warehouse inventory, storage zones and bins, stock movements, orders, vehicles, drivers, routes, shipments, shipment tracking, and transportation management. API routers expose operations, repository classes group data-access logic, and a Database abstraction provides PostgreSQL access.

Layer / component	Responsibility evidenced in the baseline
FastAPI routers	Expose HTTP endpoints for warehouse, inventory, order, and transportation operations.
Repository classes	Group persistence operations for warehouses, inventory, orders, and transportation.
Database abstraction	Manages database connectivity and query/transaction operations.
Pydantic models	Represent and validate request/response data, including transportation-related fields.
PostgreSQL	Stores the application’s logistics records.

3. Environment and Reproducibility

Component	Recorded environment
Python	3.11.5
PostgreSQL	18.6
Java	OpenJDK/Temurin 21.0.1 LTS
JavaScript runtime	Node.js 22.15.1
C++ compiler	MinGW.org GCC 6.3.0; C++17 with -O2

The WTMS backend uses Python/FastAPI and PostgreSQL. The four small language examples demonstrate a pattern independently; they do not execute the complete backend in each language.

4. Part 1 — Baseline Pattern Analysis

The primary confirmed design pattern is the Repository Pattern. Source-code analysis confirmed four repository instances: WarehouseRepository, InventoryRepository, OrderRepository, and TransportationRepository. These classes group database operations behind repository objects rather than requiring every API endpoint to contain raw persistence logic.

Pattern / approach	Baseline finding	Treatment in this report
Repository Pattern	Confirmed through four repository instances.	Counted as the primary confirmed pattern.
FastAPI dependency injection	Inspected routes use dependency injection for repository/database dependencies.	Supporting mechanism; not counted as a separate GoF pattern instance.
Database abstraction	A Database class provides connection/query/transaction facilities.	Supporting architecture; not automatically labelled Singleton.
Layered architecture	Routers, repositories, and database access have distinct roles.	Architectural approach, not a GoF pattern count.
Strategy / Observer / State / Factory	No sufficient baseline evidence established the explicit pattern structures.	Treated as possible evolution patterns, not existing baseline patterns.

4.1 Repository Pattern evidence

WarehouseRepository, InventoryRepository, OrderRepository, and TransportationRepository group data-access operations for their domains. API routes use repository objects to carry out persistence work. This separation can make database operations easier to locate and reduce duplication across endpoints. However, concrete repository classes do not prove that all repositories implement a shared interface; the baseline UML therefore does not invent an IRepository abstraction.

4.2 Limitations observed

TransportationRepository combines vehicle, driver, route, and shipment responsibilities. This may be convenient in a small application, but can become a maintenance concern as delivery-provider integrations, route policies, and tracking requirements grow. Dependency injection is not uniformly used across every inspected module, so the report avoids claiming that every component is wired through the same mechanism.

4.3 Baseline UML description

The baseline relationship can be described as: API client → FastAPI application → domain routers → concrete repository classes → Database abstraction → PostgreSQL. FastAPI Depends() supplies dependencies in inspected routes. The diagram should show concrete classes supported by source evidence and should not add unverified interfaces or event classes.

5. Part 2 — Design Evolution

An important baseline fact is that multi-warehouse storage already exists. Four warehouse records were verified. Consequently, the change is not to add multi-warehouse storage from scratch; it is to support more intelligent warehouse selection and extend delivery, tracking, and routing capabilities.

Change	What was added	Evidence / verification recorded
Baseline multi-warehouse support	No new storage mechanism required; baseline already supported multiple warehouses.	Four warehouse records verified.
Change 1: Delivery partners	Delivery-partner persistence and shipment delivery_partner_id support.	Partner creation HTTP 201; retrieval HTTP 200; shipment creation with partner ID HTTP 201.
Change 2: Tracking	Tracking records with shipment ID, latitude, longitude, location, status, and timestamp.	Tracking creation HTTP 201; history retrieval exercised.
Change 3: Route changes	Shipment route-history storage and a transactional route-change operation.	Valid route change and history verified; invalid foreign-key route rejected.

5.1 Delivery-partner evolution

The new delivery-partner data and shipment association provide a persistent representation of the partner used for a shipment. An Adapter Pattern would be appropriate if external providers expose incompatible APIs. The adapter itself should only be described as implemented if provider-specific clients are wrapped behind a common application-facing interface; database partner records alone do not prove an Adapter Pattern.

5.2 Shipment tracking evolution

The tracking extension stores location and status history rather than relying only on current shipment status. This provides a basis for an auditable sequence of updates. A full Observer implementation additionally requires a publisher/subscriber mechanism and actual notification consumers. Persisting tracking rows alone is not sufficient evidence of a complete Observer Pattern or push-based real-time updates.

5.3 Route-change evolution

The route-change operation updates the shipment route and records the change in history within a transaction. The new route assignment and audit record should not disagree. Invalid route references are expected to fail rather than silently produce inconsistent records.

6. Pattern Evolution Summary

Pattern / mechanism	Baseline	Evolution status	Explanation
Repository Pattern	Present	Preserved / extended	Existing repositories remain the data-access boundary as new persistence operations are added.
Dependency injection	Present in inspected routes	Preserved	Can supply new services and repositories.
Layered architecture	Present	Extended	New service logic can be added without removing existing layers.
Delivery-partner persistence	Not established as baseline capability	Newly introduced	Partner records and a shipment partner reference were added.
Tracking history	No dedicated tracking history established	Newly introduced	Tracking records can be stored and retrieved.
Route-change history	No dedicated route history established	Newly introduced	Route assignment changes are audited transactionally.
Strategy	No explicit baseline hierarchy established	Proposed unless implemented	Useful for interchangeable warehouse, partner-selection, or route-selection algorithms.
Observer	No explicit publisher/subscriber mechanism established	Proposed unless implemented	Useful when tracking events notify independent consumers.
State	Status represented as data	Proposed unless implemented	Useful when lifecycle transitions have state-specific rules.

The statuses are evidence-based. A new table or endpoint does not automatically imply that Strategy, Observer, or State has been implemented. These patterns require relevant classes and collaborations to exist in code and to be exercised by tests.

7. Part 3 — Cross-Language Implementation

The Repository Pattern was implemented independently in Python, Java, JavaScript, and C++. Each demo performs equivalent operations: save a shipment, find a shipment by ID, retrieve all shipments, and verify expected results. All four implementations passed their functional tests.

Language	LOC	Cyclomatic complexity	Median process time	Test
Python	45	1	93.486 ms	PASS
Java	74	3	81.214 ms	PASS
JavaScript	49	3	46.009 ms	PASS
C++	65	3	14.209 ms	PASS

7.1 Interpretation of the comparison

The programs implement the same basic repository responsibilities, but languages express data structures, typing, abstraction, and runtime execution differently. Python is the shortest example and has the lowest recorded cyclomatic complexity. These values describe the selected demo files only; they are not measurements of the full WTMS backend.

The observed process-level median was lowest for the C++ demo and highest for Python in this local run. This is not a general language-performance ranking: process startup, interpreter/runtime initialization, compiler options, machine state, and the small workload all influence the result. The timing data records this experiment, not a universal conclusion.

7.2 Before-and-after concept

Before: a repository object groups shipment data operations, with the caller depending on its concrete API.
After: the same repository responsibilities are demonstrated in four languages with a functional test verifying save, lookup, and retrieval. The comparison uses equivalent behaviour rather than unrelated examples.

8. Measurement Methodology

For each implementation, compilation was performed where required. One warm-up execution was followed by five measured executions. PowerShell Measure-Command recorded process-level elapsed time, and the median of the five measured runs was used as the summary statistic. Compilation was not part of repeated runtime measurements where the executable had already been built.

The timing values include process startup overhead and are specific to the local environment. They are process-level results, not isolated function-call timings. The procedure does not eliminate every source of operating-system noise.

9. Testing and Verification

Test area	Verification recorded	What it supports
Delivery partner	Create, retrieve, and shipment association requests returned expected HTTP statuses.	Partner association API and persistence path was exercised.
Tracking	Tracking record creation and history retrieval were exercised.	Tracking data can be persisted and queried.
Route change	Valid route change and history checked; invalid route reference rejected.	Route update has validation and transactional history behaviour.
Python demo	Functional test passed.	Save/find/list behaviour matched expected results.
Java demo	Functional test passed.	Save/find/list behaviour matched expected results.
JavaScript demo	Functional test passed.	Save/find/list behaviour matched expected results.
C++ demo	Functional test passed.	Save/find/list behaviour matched expected results.

These recorded results are separate from proposed tests for patterns that remain design recommendations. A test should only be marked passed after execution and evidence capture. In particular, a complete WebSocket-based Observer system, external delivery-provider adapters, and a formal State hierarchy must not be claimed as tested unless their implementation and test evidence are available.

10. Limitations and Threats to Validity

• The four cross-language programs are small representative examples, not full logistics applications.
• Timing includes process startup overhead and depends on the local machine and runtime environment.
• Cyclomatic complexity and line counts depend on file boundaries and counting rules; comparisons are meaningful only when rules are applied consistently.
• The baseline pattern inventory is conservative and counts only structures supported by source evidence.
• Persisted tracking history is not the same as a live push-notification system. Partner persistence is not by itself an Adapter Pattern, and route-history persistence is not by itself a Strategy Pattern.
• Verified results are kept separate from recommended future refactorings.

11. Future Work

Future improvements should solve real requirements rather than add patterns for their own sake. A warehouse-selection Strategy could separate ranking policies after eligibility constraints are enforced. Adapter is appropriate when real provider APIs differ. Observer is appropriate when tracking events must reach independent consumers, potentially through WebSockets or a message broker. State is appropriate if lifecycle rules justify explicit state objects or a centralized transition policy. These additions should include unit and integration tests, migration evidence, and before/after measurements.

12. Conclusion

The WTMS baseline demonstrates a Repository-based data-access architecture with four confirmed repository instances. Multi-warehouse capability already exists, so the evolution work focuses on intelligent selection and additional logistics features rather than inventing support that was already present. Delivery-partner association, shipment tracking history, and transactional route-change history extend capabilities while retaining the repository abstraction. The four language demonstrations show equivalent Repository Pattern behaviour and passed their functional tests. Results are documented with a consistent process-level timing method and interpreted conservatively. The central lesson is to identify patterns from evidence, introduce new patterns only when they solve a real design problem, and support claims with executable tests and reproducible measurements.

Appendix A — Evidence and Supporting Files

Pattern evidence is maintained in patterns.csv and pattern_counts.csv, evolution records in patterns_evolution.csv, and cross-language results in cross_language.csv. Part 1, Part 2, and Part 3 evidence files and UML diagrams are stored under the CaseStudy directory. The README should record source revision, environment versions, run/test commands, file and complexity counting rules, and features that remain proposals.